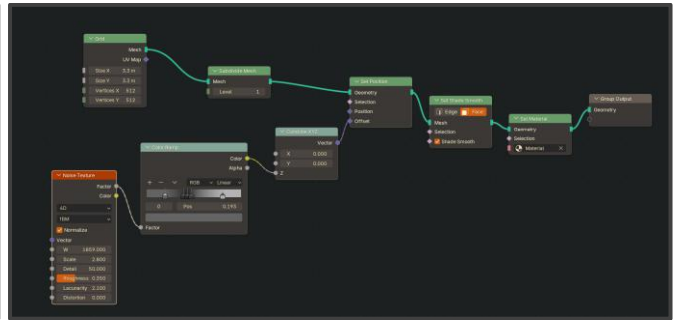
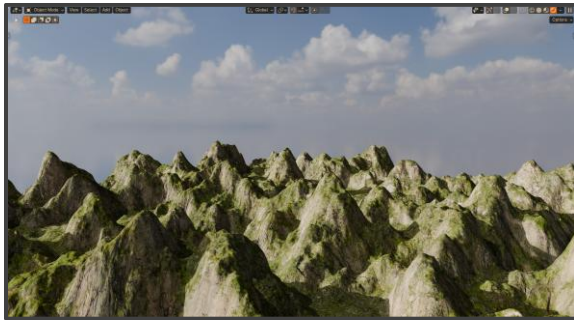


INFINITE WORLD GENERATOR PROGRESS REPORT

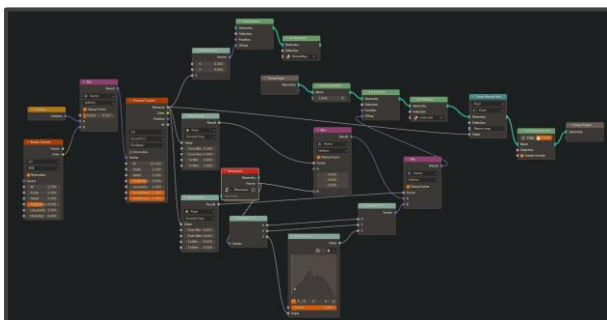
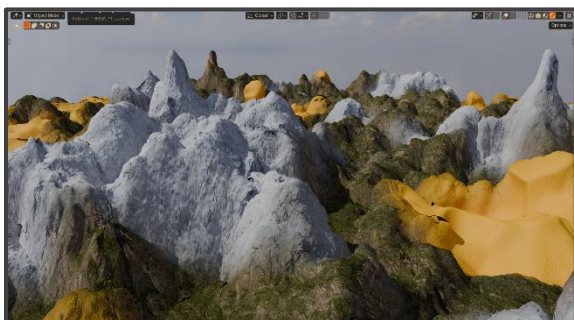
Week 1: Blender & Procedural Thinking

- **What was done:** The first week introduced Blender, Geometry Nodes, and concepts of procedural generation. The goal was to shift from manually designing environments to building automated systems that generate terrain and landscapes. Topics covered included noise functions, heightmaps, terrain displacement, and the balance between too random and too uniform.
- **How it was implemented:** A procedural terrain generator was created using Geometry Nodes. The setup generated mountains, hills, and valleys. It was implemented with custom controls that allowed for adjustments to the noise scale, height intensity, and random seed variation.



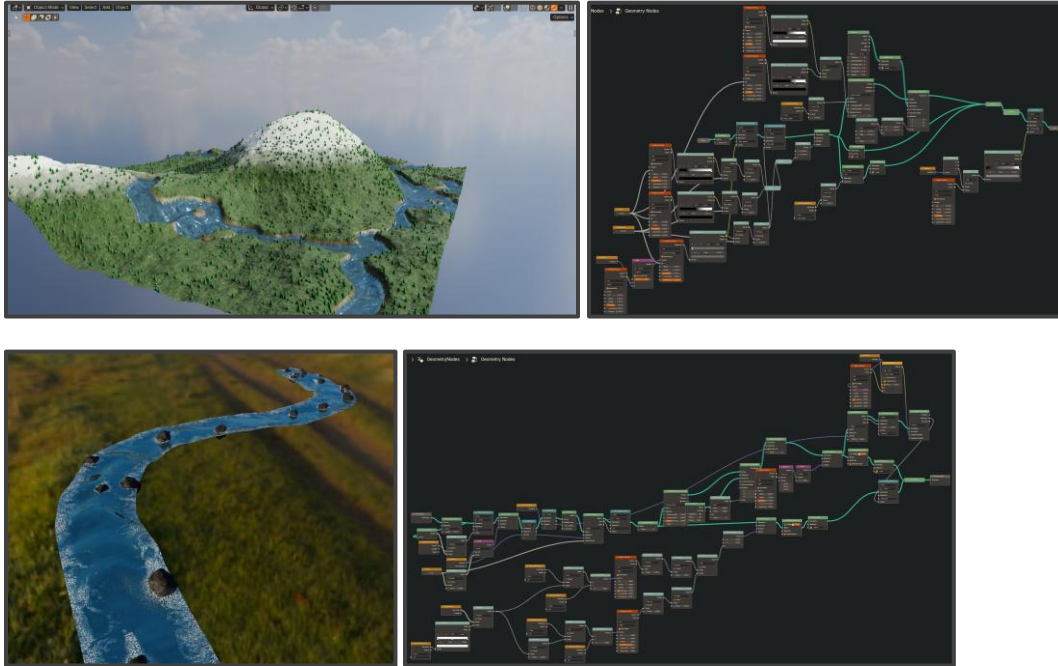
Week 2: Terrain Systems and Biomes

- **What was done:** The project expanded the base terrain into a biome-aware world featuring distinct regions and material blending. The focus was on learning terrain masking, and biome logic based on elevation and slope.
- **How it was implemented:** A multi-biome system was developed containing three biomes: desert, mountains, and snow. This was achieved using the mountain biome from last week, and a desert biome made using Voronoi noise for dunes. Biome masks with height detection logic were utilized to enforce clear distribution rules for textures and height maps.



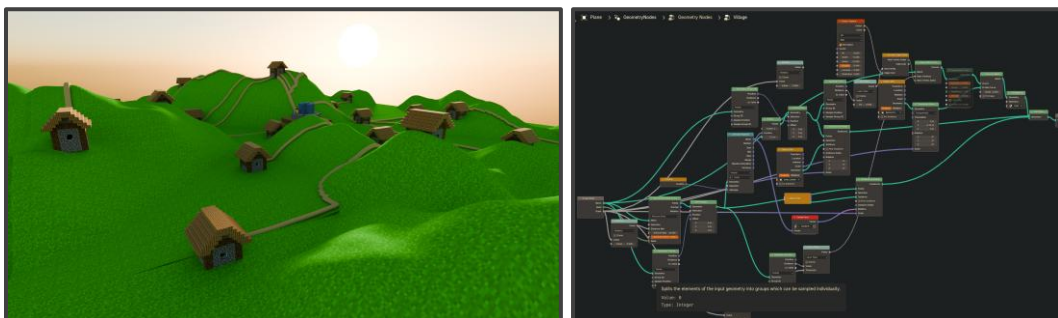
Week 3: Water, Vegetation & Scatter Systems

- **What was done:** This week focused on making the generated world feel alive by adding natural elements like rivers, lakes, forests, and vegetation zones.
- **How it was implemented:** Vegetation, including trees, grass, and rocks, was scattered across the terrain using Geometry Nodes for distributing points on the terrain and instancing the trees on it. This scattering was controlled using density masks. An animated river was also made procedurally.



Week 4: Roads, Villages and Scene Polish

- **What was done:** Civilization was brought into the procedural world through the addition of roads, settlements, and scattered buildings. The environment was polished with atmospheric lighting and final rendering techniques.
- **How it was implemented:** Roads were procedurally generated using curves. The automated placement of settlements and buildings was done by first randomly scattering houses, then selecting the center of those houses as the town center, then finding paths between the center and houses with random costs. The deliverable was showcased using a final render and camera flythrough.



Week 5: Python Inside Blender

- **What was done:** The project transitioned to using Python to automate workflows and programmatically control the built Geometry Nodes. The focus was on learning basic Python syntax, data structures, and navigating the Blender Python API (bpy).
- **How it was implemented:** A Python script was written to act as a Random Terrain Preset Generator. This script first created a low-poly tree then manipulated exposed Geometry Nodes parameters (like seed, scale and density) to output a reproducible terrain setup. The script also printed the resulting preset variables to the console.

Week 6: Chunk Based Generation using Python

- **What was done:** This time python was used to make terrain what was divided in chunks. Also all previous thing were also present like the tree scattering and random terrain.
- **How it was implemented:** This system divided the infinite world into chunks, loading nearby areas to improve performance and memory usage. All the parameters for the terrain generation like the seed, the tree object and chunk coordinates were procedurally set so they could be set using the code. All the chunks also used the same node tree to save memory. The generated chunk data was stored and serialized using JSON files.

Week 7: Building the Infinite World Generator Add-on

- **What was done:** The standalone Python scripts and procedural algorithms were combined into a functional Blender add-on. This creates a distributable thing which anyone can use to manage presets and generate worlds without modifying source code.
- **How it was implemented:** A custom UI panel was built into Blender's Sidebar (N-Panel) using the Blender Python API. Adjustable parameters such as seed, terrain size, mountain strength and tree density were exposed using Blender Properties. Blender Operators were registered to create interactive buttons for "Generate World" and "Randomize Seed". The project was organized into a modular structure (separating properties, panels, vegetation, and generator logic) where Python acted as the controller feeding input into the Geometry Nodes pipeline.

